

Test Execution Logging Design

Architecture, collection, storage, processing, analysis, and lifecycle of SoC validation logs

Generated from source specification

2026-10-05

Contents

Test Execution Logging Design	2
1. Purpose	2
2. Scope	2
3. Design Goals	2
3.1 Functional Requirements	3
3.2 Recommended event fields	3
3.3 Log levels	3
3.4 Result states	3
3.5 Retry visibility	4
3.6 Cleanup and Safe-State Logging	4
4. Logging Architecture	4
5. What Should Be Logged?	4
5.1 Test Framework Logs	4
5.2 Test Action Logs	5
5.3 SoC Interface Logs	5
UART	5
SPI	5
I2C	6
JTAG/SWD	6
5.4 Firmware Logs	6
5.5 Hardware / Board Logs	6
5.6 Environment Information	7
6. Execution ID	7
7. Test ID	8
8. Log Entry Structure	8
9. Recommended Event Schema	9
10. Log Levels	9
11. Storage Architecture	9
12. Long-Term Storage	10
12.1 Artifact Storage	10
13. Failure Classification	10
14. Failure Analysis Pipeline	11
15. Test Steps	11

16. Retention Policy	12
17. Log Rotation	12
18. Security and Sensitive Data	13
19. Recommended Initial Architecture	13
20. Key Design Principles	13
20.1 Preserve Raw Evidence	13
20.2 Every Event Must Be Traceable	13
20.3 Test Results and Logs Are Different Things	14
20.4 Logs Must Support Failure Reconstruction	14
20.5 Logging Must Be Configurable	14
20.6 Logging Must Be Low Overhead	14
21. Final Architectural Principle	14

Test Execution Logging Design

1. Purpose

This document defines the architecture, format, collection, storage, processing, analysis, and life-cycle of test execution logs generated during SoC validation.

The objective is to provide:

- complete traceability of every test execution;
- a consistent logging mechanism across different test types;
- correlation between test actions and SoC responses;
- preservation of raw diagnostic information;
- machine-readable data for automated analysis;
- human-readable logs for debugging;
- integration with CI/CD and test reports;
- reproducibility of failures;
- long-term historical analysis and regression detection.

The logging system should support both:

- **interactive/local execution**, and
- **automated CI execution**.

2. Scope

The logging architecture should cover the complete execution chain: {mermaid} flowchart TD A["Test Specification"] --> B["Test Runner
(pytest)"] B --> C["Test Framework"] C --> D["UART"] C --> E["JTAG / SWD"] C --> F["Board Control"] D --> G["SoC"] E --> G F --> G

{mermaid} flowchart TD A["SoC"] --> B["Measurements / Responses"] B --> C["Log Collector"] C --> D["Raw Logs"] C --> E["Structured Events"] C --> F["Artifacts"] D --> G["Analysis / Reporting"] E --> G F --> G G --> H["JUnit"] G --> I["HTML / PDF"] G --> J["Database"]

3. Design Goals

These should be treated as explicit requirements.

3.1 Functional Requirements

->table<- | ID | Requirement | |:-|:-| | LOG-001 | Every test execution shall have a unique Execution ID | | LOG-002 | Every test case shall have a unique Test ID | | LOG-003 | All log entries shall be associated with an Execution ID | | LOG-004 | Logs shall support long-term historical analysis | | LOG-005 | Raw interface output shall be preserved | | LOG-006 | Log entries shall contain timestamps | | LOG-007 | Test actions shall be logged | | LOG-008 | Test verdicts shall be logged | | LOG-009 | Failures shall contain sufficient diagnostic information | | LOG-010 | Logs shall be machine-readable | | LOG-011 | Logs shall be human-readable | | LOG-012 | Logs shall be usable outside the CI environment | | LOG-013 | Test artifacts shall be associated with the execution | | LOG-014 | Logging shall have minimal impact on test execution |

3.2 Recommended event fields

Each machine-readable event should include, where applicable:

- Timestamp
- Log level
- Component
- Action
- Result
- Test ID
- Human-readable message
- Structured context

Example:

```
{
  "timestamp": "2026-09-18T14:25:32.456+02:00",
  "elapsed_ms": 10543,
  "level": "INFO",
  "component": "Keithley2280",
  "action": "set_voltage",
  "parameters": {
    "channel": 1,
    "voltage_v": 1.8
  },
  "result": "SUCCESS",
  "test_id": "TEST-POWER-004"
}
```

3.3 Log levels

Use a clear and consistently enforced definition:

TRACE	Very detailed protocol traffic, register access, or SCPI traffic
DEBUG	Internal state, decisions, variables, and diagnostic context
INFO	Normal test milestones and successful operations
WARNING	Unexpected but recoverable condition, fallback, or retry
ERROR	Operation or step failed, but cleanup or partial continuation is possible
CRITICAL	Unsafe or unrecoverable condition requiring immediate abort

Allow log-level configuration globally and per component so noisy protocol traces can be enabled selectively.

3.4 Result states

Result state describes the outcome of an execution, test, step, assertion, or operation. ->table<- | Result | Meaning | |:-|:-| | PASS | Expected behavior was verified without recovery | | PASS_WITH_RETRIES | Final expectation passed after one or more failed attempts | | FAIL | Observed behavior did not meet an expected result | | ERROR | Framework, equipment, or environment prevented valid execution | | SKIP | Execution was intentionally not performed | | BLOCKED | A required prerequisite was not satisfied | | ABORTED | Execution was intentionally stopped or ended for safety | | INCOMPLETE | Execution ended without a reliable final state |

Keeping **FAIL** separate from **ERROR** prevents product failures from being mixed with test-infrastructure problems.

3.5 Retry visibility

Every attempt should be logged. Reporting only the final successful attempt can hide instability.

A retry record should identify:

- operation and related step;
- attempt number and configured limit;
- failure reason;
- delay or recovery action;
- final outcome.

Retry able conditions should be defined explicitly. Mandatory failures should not be silently converted into successful results.

3.6 Cleanup and Safe-State Logging

Cleanup should be treated as a formal execution phase rather than an unlogged final action.

Log:

- cleanup start and completion;
 - resources released;
 - device and equipment target states;
 - observed final states;
 - verification outcome;
 - cleanup errors and emergency fallback actions.
-

4. Logging Architecture

The logging system should be separated into four logical layers. `{mermaid} flowchart TD A["Test Execution Layer
pytest / test cases / fixtures / assertions"] --> B["Automation Layer
Test actions / drivers / framework events"] B --> C["UART Log"] B --> D["JTAG / SWD Logs"] B --> E["Board HW Logs"] C --> F["Log Collection Layer"] D --> F E --> F F --> G["Raw Logs + Structured Events + Metadata"]`

```
{mermaid} flowchart TD
    A["UART Log"] --> D["Log Collection Layer"]
    B["JTAG / SWD Logs"] --> D
    C["Board HW Logs"] --> D
    D --> E["Storage / Analysis Layer"]
    E --> F["Files / DB / CI Artifacts / Reports"]
```

This separation is important because **raw logs and interpreted logs should not be the same thing**.

5. What Should Be Logged?

Logs should be divided into well-defined categories.

5.1 Test Framework Logs

Generated by pytest or the test framework.

Examples:

- TEST_START
- TEST_STEP
- TEST_PASS
- TEST_FAIL
- TEST_SKIP
- TEST_ERROR
- TEST_END

Example:

```
10:21:15.120 INFO TEST_START
10:21:15.124 INFO test_id=SOC_UART_001
10:21:15.125 INFO step="Reset DUT"
10:21:17.342 INFO step="Wait for boot"
10:21:18.002 INFO step="Send UART command"
10:21:18.103 INFO step="Validate response"
10:21:18.104 INFO TEST_PASS
```

5.2 Test Action Logs

Every significant action performed by the automation framework should be logged.

Examples:

- Power ON
- Power OFF
- Reset
- Flash firmware
- Open UART
- Close UART
- Send command
- Read response
- Configure peripheral
- Read register
- Write register
- Start measurement
- Stop measurement
- Attach debugger
- Detach debugger

Example:

```
ACTION
execution_id=EXE-20260918-001
test_id=SOC_UART_001
action=UART_SEND
interface=UART0
data="AT+STATUS"
```

5.3 SoC Interface Logs

These capture communication with the DUT.

UART

Recommended fields:

```
timestamp
interface
direction
data
baudrate
```

Example:

```
12:31:02.120 UART0 TX  "status"
12:31:02.145 UART0 RX  "STATUS: READY"
```

SPI

Example:

SPIO
TX: 9F 00 00 00
RX: EF 40 18 00

I2C

Example:

I2C0
ADDR=0x50
WRITE=00 10
READ=AB CD

JTAG/SWD

Examples of events:

- connect
 - reset
 - halt
 - resume
 - register read
 - memory read
 - memory write
-

5.4 Firmware Logs

Firmware-generated messages should be preserved separately from automation logs.

Example:

```
[BOOT] Starting system
[CLK] PLL configured
[MEM] DDR initialization OK
[UART] Driver initialized
[APP] Application started
```

The source of the message should be retained:

```
source = DUT
interface = UART0
```

Do not simply merge firmware output into the Python log and discard its origin.

5.5 Hardware / Board Logs

If the test infrastructure controls the physical board, capture:

- power supply state;
- voltage;
- current;
- temperature;
- reset state;
- relay state;
- board controller messages;
- power-cycle events;
- external instrumentation.

Example:

```
BOARD_POWER_ON
voltage=3.30V
current=142mA
timestamp=2026-09-18T12:31:00.123Z
```

For long-running tests, this information can be extremely valuable.

5.6 Environment Information

Every execution should capture the environment in which the test was executed.

Example:

```
execution:
  execution_id: EXE-20260918-001
  timestamp: 2026-09-18T12:30:00Z

test:
  suite: regression
  test_id: SOC_UART_001
  test_version: abc1234

dut:
  device: XYZ123
  silicon_revision: B1
  serial_number: DUT-00017
  board_revision: REV_C

firmware:
  version: 2.4.1
  git_commit: 8f23ab1
  build_id: BUILD-12345

environment:
  runner: Jenkins
  runner_id: runner-04
  python_version: 3.12
  pytest_version: 8.x
  os: Ubuntu 24.04

tools:
  debugger: J-Link
  debugger_version: x.x.x
```

This information is critical for failure reproduction.

A failure six months later is almost impossible to reproduce if the only information available is:

```
test_uart FAILED
```

6. Execution ID

The system should introduce an **Execution ID** as the primary correlation mechanism.

Example:

EXE-20260918-153045-7F32

Everything generated during that execution should reference it.

Execution

```
Test Case 001
  Python log
  UART log
  JTAG log
  screenshots
```

```
Test Case 002
  Python log
  UART log
```

```
Test Case 003
  Python log
  UART log
  power measurements
```

The Execution ID should remain stable for the entire test run.

7. Test ID

Each test should have a stable identifier.

Examples:

```
SOC-B00T-001
SOC-UART-001
SOC-SPI-014
SOC-INT-023
SOC-PWR-007
```

Avoid using the Python function name as the only identifier.

For example:

```
def test_uart_command():
    ...
```

can be renamed.

Instead, the test should have a stable logical identifier:

```
@pytest.mark.test_id("SOC-UART-001")
def test_uart_command():
    ...
```

The stable Test ID can then be used for historical analysis.

8. Log Entry Structure

The canonical machine-readable format should be **JSON Lines (JSONL)**.

One event should correspond to one JSON object.

Example:

```
{
  "timestamp": "2026-09-18T12:31:02.120Z",
  "execution_id": "EXE-20260918-001",
  "test_id": "SOC-UART-001",
  "source": "test_framework",
  "level": "INFO",
  "event": "TEST_START",
  "message": "UART test started"
}
```

Another example:

```
{
  "timestamp": "2026-09-18T12:31:05.120Z",
  "execution_id": "EXE-20260918-001",
  "test_id": "SOC-UART-001",
  "source": "uart",

```

```

"interface": "UART0",
"direction": "TX",
"event": "DATA",
"data": "status\\r\\n"
}

```

And:

```

{
  "timestamp": "2026-09-18T12:31:05.145Z",
  "execution_id": "EXE-20260918-001",
  "test_id": "SOC-UART-001",
  "source": "uart",
  "interface": "UART0",
  "direction": "RX",
  "event": "DATA",
  "data": "STATUS: READY\\r\\n"
}

```

9. Recommended Event Schema

A common event schema should be defined for all log sources.

Recommended mandatory fields: ->table<- | Field | Description | |:-|:-|
timestamp | Event timestamp in UTC | **execution_id** | Unique execution identifier | **source** | Component that generated the event | **level** | Severity | **event** | Event type |

Recommended contextual fields: ->table<- | Field | Description | |:-|:-|
test_id | Test case identifier | **interface** | DUT interface | **dut_id** | DUT identifier | **board_id** | Board identifier | **message** | Human-readable description | **data** | Raw or structured event data |

Optional fields should be added based on the source.

10. Log Levels

Log levels should be defined centrally. ->table<- | Level | Purpose | |:-|:-|
TRACE | Extremely detailed diagnostic information | DEBUG | Developer/debug information | INFO | Normal execution information | WARNING | Unexpected but non-fatal condition | ERROR | Test/framework error | CRITICAL | Infrastructure/system failure |

Example:

```

INFO      Flashing firmware
INFO      Firmware flash completed
DEBUG     Waiting for boot message
INFO      DUT boot completed
WARNING   Boot took longer than expected
ERROR     UART response timeout

```

11. Storage Architecture

For the initial implementation, a filesystem-based artifact structure is sufficient.

A centralized logging platform such as Elasticsearch/Kibana should only be introduced if log volume and search requirements justify it.

Recommended architecture: {mermaid} flowchart TD
A["CI Runner"] --> B["Execution Directory"]
B --> C["raw_logs/"] B --> D["events/"] B --> E["artifacts/"] C --> F["UART.log
JTAG.log"]
D --> G["events.jsonl
results.json"] E --> H["firmware.bin
dump.bin
trace.csv"]

All files should be associated with the same Execution ID.

12. Long-Term Storage

The storage architecture should distinguish between large artifacts and searchable metadata. {mermaid} flowchart TD A["Execution"] --> B["Artifact Storage"] A --> C["Result Database"] B --> D["Raw logs / files
Dumps / Measurements"] C --> E["Test results
Metadata / Statistics"]

12.1 Artifact Storage

Stores large objects:

- raw UART logs;
- JTAG logs;
- firmware;
- memory dumps;
- traces;
- screenshots;
- power measurements.

Large raw UART/JTAG logs should generally remain in artifact storage rather than being stored directly in a relational database.

13. Failure Classification

Test results should contain more than a simple PASS/FAIL verdict.

Recommended failure categories:

- TEST_FAILURE
- ASSERTION_FAILURE
- TIMEOUT
- UART_FAILURE
- JTAG_FAILURE
- FLASH_FAILURE
- BOOT_FAILURE
- DUT_CRASH
- DUT_HANG
- POWER_FAILURE
- COMMUNICATION_FAILURE
- INFRASTRUCTURE_FAILURE
- ENVIRONMENT_FAILURE
- UNKNOWN

Example regression summary:

Regression summary

Total:	2450
PASS:	2301
FAIL:	91
INFRA_FAILURE:	32
TIMEOUT:	18
BOOT_FAILURE:	14
UART_FAILURE:	9
DUT_CRASH:	7
OTHER:	11

This is significantly more useful than a simple pass/fail count.

14. Failure Analysis Pipeline

The system should contain an automated analysis stage.

Raw Logs

Parser

Failure Detection

- Timeout
- Assertion
- Boot failure
- Crash
- Communication
- Infrastructure

Failure Summary

Test Report

Example failure report:

TEST FAILED

Test:
SOC-UART-014

Failure:
UART response timeout

Expected:
"READY"

Observed:
No response

DUT state:
Boot completed

Last DUT message:
"Initializing UART driver..."

Likely category:
DUT / UART initialization

Artifacts:
UART log
JTAG log
Firmware
Memory dump

15. Test Steps

Explicit test steps are recommended for all complex tests.

Example:

TEST: SOC-UART-001

```

STEP-001  Power on DUT
STEP-002  Flash firmware
STEP-003  Reset DUT
STEP-004  Wait for boot
STEP-005  Send command
STEP-006  Validate response
STEP-007  Power off DUT

```

A log entry can therefore look like:

```

{
  "timestamp": "2026-09-18T12:31:05.120Z",
  "execution_id": "EXE-20260918-001",
  "test_id": "SOC-UART-001",
  "step_id": "STEP-005",
  "source": "test_framework",
  "level": "INFO",
  "event": "STEP_START",
  "message": "Send UART command"
}

```

16. Retention Policy

Retention requirements should be explicitly defined.

Example: ->table<- | Artifact | Example Retention | | Test result metadata |
 1–2 years | | Failed test logs | 1 year | | Successful raw logs | 30–90 days | Firmware/build metadata | 1–2 years |
 | Crash dumps | 1 year | | Regression reports | 1–2 years | | Debug traces | 30 days |

The exact values should be agreed with the project.

The retention policy should also define:

- maximum log size;
 - compression;
 - archive format;
 - deletion policy;
 - ownership;
 - storage limits.
-

17. Log Rotation

Long-running SoC tests can generate very large logs.

Log rotation should therefore be supported.

Example:

```

uart.log
uart.log.001
uart.log.002
uart.log.003
uart.log.current

```

Rotation can be triggered by:

- file size;
- elapsed time;
- test execution;
- DUT reboot.

Example policy:

Maximum file size: 100 MB

Compression: gzip

The actual limits should be configurable.

18. Security and Sensitive Data

The logging system must define what information is permitted in logs.

Potentially sensitive information includes:

- credentials;
- authentication tokens;
- private keys;
- production secrets;
- customer information;
- proprietary information.

The logger should support masking.

Example:

```
username=test_user
password=*****
token=*****
```

Sensitive information should not be written to raw logs unless explicitly required and appropriately protected.

19. Recommended Initial Architecture

For a Python/pytest-based SoC automation environment, the recommended initial architecture is: {mermaid} flowchart TD A["pytest"] --> B["Logging Manager"] B --> C["Framework Events"] B --> D["UART Driver"] B --> E["JTAG / SWD"] C --> F["Event Collector"] D --> F E --> F

```
{mermaid} flowchart TD
    A["Event Collector"] --> B["Raw Log Files"]
    A --> C["JSONL"]
    B --> D["Test Analyzer"]
    C --> D
    D --> E["JUnit"]
    D --> F["JSON DB"]
    D --> G["Quarto"]
    E --> H["Historical Analysis"]
    F --> H
    G --> H
```

20. Key Design Principles

The following principles should be treated as architectural decisions.

20.1 Preserve Raw Evidence

Raw evidence shall never be destroyed as a result of log processing.

Raw logs must remain available even after parsing and analysis.

20.2 Every Event Must Be Traceable

Every event should be traceable to:

Execution

DUT

Test

Step

Event

20.3 Test Results and Logs Are Different Things

A test result answers:

Did the test pass or fail?

A log answers:

What happened during execution?

The two should be linked but not treated as the same data.

20.4 Logs Must Support Failure Reconstruction

The system should provide enough information to reconstruct a failure without requiring access to the original test runner session.

20.5 Logging Must Be Configurable

Different test types have different logging requirements.

The system should support at least: ->table<- | Level | Meaning | |:-----|:-----
| | TRACE | Very detailed protocol or diagnostic information | | DEBUG | Internal
state and diagnostic context | | INFO | Normal milestones and successful operations | | WARNING | Unexpected
but recoverable condition, fallback, or retry | | ERROR | Operation or step failure where cleanup or controlled
continuation remains possible | | CRITICAL | Unsafe or unrecoverable condition requiring immediate termination
|

and source-specific configuration.

20.6 Logging Must Be Low Overhead

Logging must not materially alter the behavior being tested.

This is particularly important for timing- and performance-sensitive SoC validation.

21. Final Architectural Principle

The most important principle for the overall design is:

A test execution log shall contain sufficient information to reconstruct and diagnose the execution without requiring access to the original test runner session.

This leads to the second key principle:

Raw evidence shall be preserved independently from processed and analyzed results.

Together, these principles should drive the decisions regarding event structure, correlation IDs, storage, retention, analysis, and reporting.
